

Development of a Novel Semantic-based System Integration Framework

András Bánhalmi, Dénes Paczolay, Ádám Zoltán Végh, Gábor Antal, Vilmos Bilicki

Department of Software Engineering

University of Szeged

Szeged, Hungary

{banhalmi,pdenes,azvegh,antal,bilicki}@inf.u-szeged.hu

Abstract— It is a frequent problem in system development when various different systems with similar functionality have to be integrated together. System integration generally means accessing, sharing and exchanging data among applications. This data sharing can be achieved by implementing interfaces for communication and data exchange, and services for all the applications to be integrated. However, this solution requires a large amount of human resources for software development and refactoring. Here, a novel framework is proposed for data integration that makes the process much simpler by generating all the source code needed for querying data of an integrated system. The proposed framework is based on two ontology mediators; namely, a local ontology generated for the data source to be integrated, and a global or central ontology for the common query interface. This way, the integration process is reduced to finding automatically or semi-automatically the mapping between the concepts of local and global ontologies. If this semantic matching is performed, then all the predefined SPARQL queries of global ontology will be rewritten automatically to queries belonging to data sources to be integrated. Hence, the differences between the systems to be integrated will appear just in the DAO (Data Access Object) layer that is generated according to the ontology alignments. The proposed integration technique does not need any program coding in practice (only visual concept matching has to be done semi-automatically), so the productivity of integration software development can be improved significantly. To demonstrate our concept, our novel framework is applied to real-world integration tasks. In this paper, the framework, methodology, experiences and some measurements relating to the productivity will be presented and discussed.

Keywords- *System integration, data integration, ontology matching, local ontology, ontology mapping; SPARQL rewriting, SPARQL translation*

I. INTRODUCTION

As the size of data and the number of various services on Internet are growing, there is an increasing need to collect and integrate all the reachable semantic data for a predefined domain. Information sources to be integrated are very heterogeneous: Web pages (with or without meta information, structured XML documents), relational databases, NoSQL databases, Web services, and so on. Letting people or agents search interesting data semantically from the various forms of information sources is one of the most important purposes of WEB 3.0. For this, Web pages should be labeled with semantic information automatically, this topic being a part of the

information extraction research. Another important task is to reach the data stored in different conventional databases, which have been present for a long time, and contain a huge amount of data. This task can be handled by using semantic mediators to mirror these data stores. To describe and store semantic information, many standards were defined previously, these being the so-called semantic webs, or ontologies (RDF, OWL [1], [2]). Query languages also exist to get information from these ontologies (e.g. SPARQL [3]). After many information sources have been semantically described by ontologies, integration can be achieved by connecting these local ontologies to a global one. Here, it is supposed that the concepts of the local ontologies correspond to a similar domain which is not too broad. The global ontology can be created in several ways; it can be set by hand, or it can be generated from a set of local ontologies. In addition, it can be generated from the source code of a central application as well. When the global ontology is defined and also the local ones, then the connections among the concepts have to be found. This can be done semi-automatically by using ontology matching methods. Using the correctly specified connections, a SPARQL query of the global ontology can be rewritten to a SPARQL query of the local one, which can then be rewritten for an SQL query of the integrated database. Applying this chain of methods, a DAO (Data Access Object) layer can be generated containing the SQL queries for all the integrated data sources and for predefined SPARQL queries of the global ontology. One can see that the main task, when integrating a new data source, is the automatic generation of a corresponding local ontology and the semi-automatic matching of the new ontology and the central one.

In the following, we provide an overview of work related to the area of semantic data integration. Then a summary of the proposed framework will be given, whose framework will then be divided into parts, and these parts will be elaborated on by discussing some problems and tasks that have to be solved. To demonstrate our concept, the proposed framework will be applied to a real-world problem set: to a specific one with rich domain model and a general one with simple domain model, and some investigation will be done concerning the productivity increment in the development of system integration.

II. RELATED WORK

The integration of different relational databases and information sources using a mediator was one of the earliest techniques proposed [4]. Later, ontologies became

the most widely used mediators for data integration [5]–[7]. Earlier, a lot of different ontology definitions were employed [8]. In [9], a description is given of a framework which uses the OWL or RDF ontology description language to specify local ontologies as mediators, but in this solution all the data examples (individuals) are also included in local ontologies, not just the vocabulary of the concepts. This can be a problem when the content of a database is updated; in this case the ontology should be somehow synchronized. The D2R application is similar solution [6], which can generate RDF from a database schema. Its improved version will be used in the proposed framework [10].

In [9], just a schematic description of an integration framework is given which processes some queries, but this framework also includes all the instances; so there many open issues remain. The other important difference between the earlier ontology-based integration systems [11], [12] and the framework proposed by us is that these build a large global ontology by including all the local ones (individuals). Also, in these systems, all the relations among concepts are given using OWL language statements, but there are many relations that cannot be described in this way. In that case, over the model of the whole ontology (which includes the local ones) a complex reasoner is executed to find individuals that satisfy an expression.

Yet it is also possible that the ontologies have not been integrated into a large one; rather, the local ones were left unchanged, and just the connections of concepts were defined separately. In this “hybrid” approach the types of connections can be wider than the relations described in the OWL language.

Extended concept relations and their uses were proposed in [13]–[15]. If this solution is chosen, then a standard ontology reasoner cannot be applied to get individuals that satisfy an arbitrary expression. Instead, a query language is needed to create query expressions for the global ontology, and these queries have to be rewritten to local ontologies one at a time, using the relations defined among the concepts. This technique is known as SPARQL query rewriting [14], [16]. Another problem when a local ontology mediator is used is that the ontology queries have to be transformed to SQL expressions to get back the up-to-date data. This SPARQL2SQL problem is a complicated one because an optimized SQL query has to be created for a fair performance. Some theoretically promising solutions in this area were proposed [17], [18], and in the proposed integration framework one of them, the Quest solution was applied.

III. INTEGRATION FRAMEWORK

Fig. 1 shows the structure of the framework. The proposed framework is based on two ontologies; namely, the local one, and the global one. The role of the local ontology is that it is a mediator layer for the data instances that are stored in a relational database. These ontologies are generated in such a way that they satisfy the naming definition of ontologies. Thus, the concepts have some

semantic information which will be used to relate them to a global ontology. This local ontology can be queried; in our framework the query language is SPARQL. But the local ontology is only a vocabulary, and does not contain individuals. To get the up-to-date individuals from the database, a SPARQL to SQL rewriting module was needed. Jumping to the end of the proposed integration process, for each predefined query of global ontology an SQL query will be generated, and these queries are stored in a DAO module.

The global ontology (sometimes called the central ontology) has the role of collecting the required information from all the local ontologies. For this, the concepts in it have to cover the whole target domain. The global ontology can be created in many ways; namely, by a human expert, or generated from various ontologies already created for this domain; or even generated from a source code. To integrate the concepts of local ontologies, various relations have to be specified among the concepts of the global ontology and a local ontology. Queries written to the global ontology are rewritten corresponding to these particular relations or alignments. This is realized in the SPARQL-ReWrite module.

Based on the connections among the modules of the above-proposed framework, a new data source can be integrated into the system as follows (where the modules of this workflow are explained later on). First, a local ontology and other description files are generated from the schema of a relational database. The integration system already contains a global ontology that was generated earlier from some source, or assembled by hand. The next step is to find the alignments between the concepts of the two ontologies. This can be done semi-automatically by using a GUI like COMA¹ or an XML editor: the results of an automatic matching method have to be revised in most cases by a human. In essence, there are predefined SPARQL queries for the global ontology which are of interest for an application. The next step in system integration is that the global SPARQL queries are rewritten to local SPARQL queries, corresponding to the alignments. After, these local SPARQL queries are rewritten automatically to SQL ones; then these generated SQL queries can be collected in a DAO layer of the integration application.

To build our integration framework, many searches were performed to find ready-made open-source components and modules. As a result, for some important tasks, promising solutions were found, and these will be included in the proposed framework with some key modifications (the available solutions, like D2RQ, Quest and Alignapi are mentioned in later sections, and in the footnotes. Below, the modules of the proposed framework are described in more detail.

¹ <http://sourceforge.net/projects/coma-ce/>

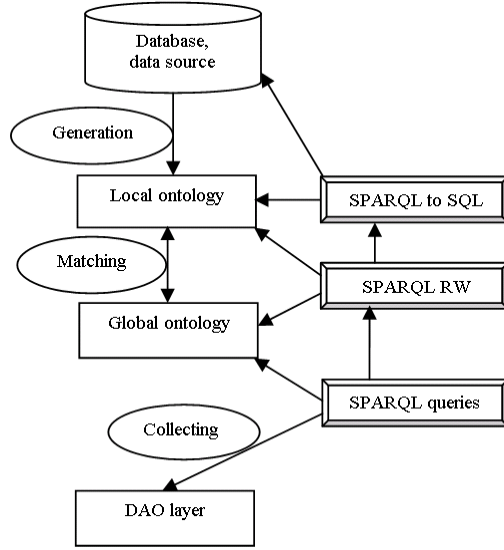


Figure 1. The structure of the proposed integration framework.

A. Local Ontology Generation

For local ontology generation, the proposed framework utilizes D2RQ open-source tool and, especially, the mapping generation process² [18]. This tool supports many relational database systems like PostgreSQL, MySQL, Oracle, Microsoft SQL server. However, some modifications were needed for two key reasons:

- The names of concepts generated by D2RQ are inconsistent in some cases. In contrast with the OWL definition, the names of different properties may be the same.
- For the purpose of SPARQL to SQL rewriting, a so-called OBDA[19] (Ontology Based Data Access) model file has to be generated, and it is expedient to generate this file in parallel with the ontology description.

The algorithm for the local ontology generation method implemented by D2RQ basically does the following:

- From the table names, classes are produced with the same name
- From attributes which are not foreign keys, data type properties are created with the name “class_attribute”
- From foreign key attributes, an object property is generated with the name class_column1_column2_otherCols.

Our modifications in the concept naming of D2RQ are based on experience that using a large, complex database with many foreign keys results in a naming conflict, because some properties get the same name. These false

examples are related to the issues when a table also has a foreign key to itself (for example, “user” table has an “admin” column which is a part of users, so it is a foreign key to same table). Other examples correspond to the naming and separation using a “_” character because when the database system used a naming convention with the same “_” separation of words, there became identical concept names. To resolve these problems, we propose to modify the generation process. Namely,

- For data type properties, use the delimiter: “_”
- For object properties, use the delimiter: “--”

However, as many experiments were done by us to handle this naming problem, we kept to the outlined solution because this one might also be helpful for similarity matching among concepts.

To generate an OBDA definition for Quest[19], some classes have to be modified to get the database connection information, to collect the names of classes, object properties, and data type properties. After, a new implementation was made for OBDA generation using information like that for the ontology generation method; namely, there is a generation rule for tables and attributes which are not foreign keys, and there is another generation rule for foreign keys that connect two tables. The idea of the OBDA model definition is to map the database columns to ontology concepts. A simple example for defining pairs of database columns and concepts is provided by the following. For a table “user (id, name, ip)” the definition of variables corresponding to columns are set by defining the SQL expression:

```
SELECT id, name, ip FROM user
```

The corresponding “virtual” definition of the individuals in the ontology is (is a tuple form):

```
{<"user/{ $id }"> rdf:type :user;
  :id $id;
  :name $name; :ip $ip}
```

For object properties and foreign keys, another simple example is given with two tables: user(id, ip, personId) and person(id, name, addr). In the generated ontology, there is an object property for personId called “user--personId” that has a domain “person”, and a range “user”. To define the connections between the ontology concepts and data columns, the OBDA for Quest definition is:

```
SELECT id, personId FROM user
```

And the ontology part of the mapping is:

```
{<"&.:user/{ $id }"> :user--personId
  <"&.:person/{ $personId }"> .}
```

B. SPARQL to SQL

The D2RQ-based, modified local ontology generation module was completed by us to create the appropriate OBDA description file for the Quest [19] reasoning tool, and to create consistent local ontologies. Using the resulting ontology and OBDA definitions, the Quest API has the appropriate methods for converting SPARQL query written to the local ontology to an SQL query corresponding to the source database. Quest is fairly robust, supports UNION, OPTIONAL and other SPARQL

² <http://d2rq.org/generate-mapping>

statements. For example OPTIONAL is needed when a query contains optional data queries along with some required properties. From these expressions outer joins (SQL) will be generated by Quest. For the rewriting process a database connection is also needed by Quest to be able to define data types of the queried columns. In the returned rows, all the column data will be converted to strings, however another column will contain the original type, so conversion is easy. It could be mentioned that Quest also supports many database systems like PostgreSQL, MySQL, Oracle, DB2 and SQL Server. It should be added, that Quest at this time is not an open-source API, but it can be used for academic research purposes. The source code is not downloadable, just the API.

C. Setting up a Central Ontology

When a new data source has to be integrated, a global ontology is already defined. However at the beginning of development of a new integration system this ontology must be set to the application domain. In several cases the global ontology is assembled by hand; in other cases it is generated using already existing ontologies. In our experiments, the aim of integration was to extract “event descriptions” from various applications, and insert them into Google Calendar. In this case, the global ontology is generated from the Java source code of the Google Calendar client API. However, from the viewpoint of the proposed framework, it is not so relevant how the global ontology is created for integration; rather, it is the process of finding relations between the concepts of global and local ontologies. One aim of our study was to measure the productivity improvement, comparing a hard-coded integration approach with the proposed semantic-based alternative approach.

D. Querying a Central Ontology

The conception of the proposed framework is a hybrid integration technique, where local and global ontologies are separated, and there is also a definition of relations among concepts in a separate file. The global ontology can be queried using the SPARQL language, and this query will be rewritten to another SPARQL query corresponding to a local ontology. To achieve this, two important tasks have to be solved: matching the ontologies, and rewriting SPARQL queries using the alignments.

E. Ontology Matching

The purpose of the ontology matching module is to support finding all the correspondences among the concepts of ontologies. This task should be essentially done by humans, but it is important to apply methods that find the majority of correspondences automatically. Then just the incorrect or missing alignments need to be handled by humans.

For ontology matching, several tools have been developed, and there are many open-source ones. However, these implementations can be used typically for indicating “level 0” alignments and 1-1 relations among

concepts. Hence, for the “level 0” ontology alignment we used an open-source solution that produced good results in OAEI evaluation campaigns, known as COMA CE (Community Edition). But this API is still under development; for alignment input/output a new code had to be implemented here. This project also contains a GUI that can be used to edit alignments by hand. When a more complex relation representation was needed, a new implementation in Java was necessary, which uses the Alignment API EDOAL extension. All the alignments were stored in “level 0” or “level 2 EDOAL” alignment format³ RDF files.

What the COMA matching framework and other tools available have in common is that they contain fairly complex matching methods that perform well on average, but these methods handle the matching problem somehow universally. For a better performance, novel matching methods are developed and proposed here that handle the special features of generated ontologies, and other typical naming issues occurring frequently in database resource names.

Here, the better matching methods in practice are not discussed in detail. This topic will be discussed in another report.

F. SPARQL Rewrite

For system integration, the perfect operation of this rewriting module is essential. The starting point and theory for developing this was published earlier [14], [16], [20]. However, at the moment there is no available source code which could be nested into our framework, so a new implementation was realized with some improvements.

The above-cited publications concerning the rewriting process introduce some cases for SPARQL rewriting:

- Data triple patterns (DTP): these patterns correspond to data querying.
- Schema triple patterns (STP): these triplets correspond mainly to schema, and data information is not considered.

These categories are further divided into cases where predicates, subjects, and objects have to be rewritten in triples. For all these issues, separate “visitor” classes were implemented in the module for rewriting. The implementation was carried out in the Java language, and some free API-s were utilized, namely, Jena API to handle ontologies and SPARQL queries (this contains a base “visitor” class), and the Alignment API which supports alignments.

Still, some modifications of the basic methods had to be made:

- Visitors have to support UNION and OPTIONAL statements in the process of SPARQL query rewriting.
- After the rewriting process, some of the concepts of global ontology did not have any relation with any concept of the local

³ <http://alignapi.gforge.inria.fr/edoal.html>

ontology. Hence, the triple patterns in a rewritten SPARQL query have to be filtered.

A post-processing phase is also needed when extended alignment relationships are used for concept rewriting. For example, when for a property in the global ontology, the union of many other properties is assigned, the resulting query will contain one or many large expressions containing a series of unions of triple patterns. However, from these triple patterns only one corresponds to a non-empty result set in most cases.

To make this clear, a brief example is given in the following. Suppose that a global ontology contains a “target:title” data type property with the domain of “target:Writing” class and this class has subclasses like “target:Book” or “target:Paper”. In the local ontology, however, this hierarchy does not appear, but there is a “source:book” class with a “book_title” data type property, and a Paper class with a “paper_title” property. The matching among the concepts are target:title=source:book_title UNION source:paper_title; target:Book=source:book; target:Paper=source:paper; target:Writing=source:book UNION source:paper respectively.

When titles of books are expressed as a SPARQL query for the global ontology:

```
SELECT ?title, WHERE{
    ?writing a target:Book.
    ?writing target:title ?title.
}
```

The rewriting process will give the result:

```
SELECT ?title WHERE{
    ?writing a source:book.
    {
        {?writing source:book_title ?title}
        UNION
        {?writing source:paper_title ?title}
    }
}
```

Here one can see that the “?writing” variable can only be source:book, so the second term of the union will give an empty result set corresponding to the ontologies and alignments. The example above was simple, but due to the differences between the structure of global and local ontologies, very long UNION expressions can occur.

To handle this problem, we propose a two-step algorithm. Namely,

1. In the first step, for each property in a UNION expression a search is made by processing triple patterns for collecting the possible domains and ranges of that property.
2. In the second step, a filtering method is applied to UNION expressions to just keep reasonable patterns.

Continuing with the example above, after the post-processing method we get the next triple patterns:

```
SELECT ?title WHERE{
    ?writing a source:book.
    {?writing source:book_title ?title}
}
```

This post-processing step is important to get a less complex SPARQL query for a local ontology, and hence a simpler SQL query.

G. DAO layer, Managing Rewritten Queries

The DAO (Data Access Object) layer is responsible for hiding the differences among information sources while all of them are providing various consistent data to the integration application. In this sense, this DAO layer can handle not only relational databases, but also NoSQL data stores and information extraction modules to get the corresponding information for a query. In a DAO layer many objects can be defined for different domains for this purpose. However, in the proposed framework this layer was developed for relational databases, which has appropriate schema information for generating a local ontology. It is our further intention to expand this layer to integrate other information sources (e.g. a set of Web pages).

For relational databases the proposed DAO layer may function in two ways, depending on the integration scheme used:

- When it contains just SPARQL queries, then the rewriting chain process will generate the real SQL queries for all the integrated systems in real time.
- For each SPARQL query corresponding to the global ontology, the rewriting chain is applied, and the resulting SQL queries are stored.

Another important problem to be solved is that for code generation some type checking and conversion methods have to be predefined and well chosen during the integration phase. In a simple example: when the local ontology contains dates given in separate concepts, like day, month, and year, but the central ontology collects these data samples in a DateTime form, then a conversion function also has to be included with the alignments. However, this can also be handled automatically or semi-automatically for a set of predefined conversion functions.

IV. PRODUCTIVITY ESTIMATES

When comparing the productivity of different development solutions for a similar problem, generally the recurring subtasks should be taken into account. There are tasks that have to be done only once for any number of integration processes, but others always have to be done when there is a new integration problem. Here it is supposed that a task performed just once is not too complex, and does not require much time to pass over it.

To estimate the productivity, the measure applied was the combined effort (amount of time) of all persons required to fulfill a task: person-hour (p. h.) is the measure used here. Measuring other possible utilized resources was ignored here.

The first concrete integration problem was the following. Some event-processing or event-displaying applications were collected, e.g. calendar and booking applications, which totaled 16. The task was to collect

event information from them to insert into the Google Calendar. Another application has a richer domain model: this is a POS printing application, which seeks to integrate data about store chains and promotion printing areas.

To solve these in the proposed integration framework, for the various applications local ontologies were generated, and from the source code of Google Calendar a global one was generated. In the POS printing problem, the global ontology was set by involving humans, taking into account the concepts corresponding to sales, promotion, and printing. Accordingly to the proposed framework, when the alignments are set properly between a local ontology and the global one, an SQL query is generated for the predefined SPARQL queries.

The other common solution is when, for all the applications collected, a development is needed which takes from its database the appropriate data and inserts it into the central database. For the “event task”, the Google Calendar is the integrating target, and such subtasks have to be solved like integrating the Google API into the project, rebuilding it and coding the appropriate integration.

In the first case, the proposed integration contains an automatic matching process applied for the concepts of the local ontology and the global one, and then some developers made use of their language skills and also the Internet when needed to correct the alignments. This process gave the productivity estimates shown in Table I. Also, before the alignment determination, the generation of the local ontology and central ontology is necessary, whose tasks in this case are as simple as running a batch file with some trivial parameters for connecting to the database and setting the root path of the source code of the Google Calendar API. It should be mentioned here that the central ontology generation only has to be executed once.

In the baseline case, when all the integration has to be done by further development or hard-coding, the following subtasks are distinguished, and the corresponding requested development durations are estimated (Table II): a learning phase, when the developer searches on the Internet for the Google API and the documentation for using it. After some program development, tests and code versions, a usable integration method is developed.

The POS printing task differs from the Calendar event tasks in that the central ontology is not a generated one; rather, it is assembled by experts. Whereas some connections were automatically generated by the framework, many modifications had to be made by hand, and two humans were involved in this task to define perfect relations. The time duration for each process is shown in Table III, but a comparison with the traditional hard-coding process was not possible because the original POS Printing code was not accessible. This could be another reason for using the data integration approach proposed here; when the data can be shared, but the whole software source code used to process them is not accessible.

It can be seen that in the productivity estimates there is an order of magnitude difference for an integration task.

This shows that the proposed concept makes this process much simpler, without any special development skills, and this could result not only in improvements in productivity, but in fewer mistakes being made as well.

TABLE I. PRODUCTIVITY ESTIMATES OBTAINED WHEN THE INTEGRATION FRAMEWORK IS APPLIED IN THE CALENDAR INTEGRATION TASKS.

Task	Minimum p. h.	Maximum p. h.	Average p. h.
generate local ontology	0.1	0.3	0.15
generate central ontology	0.1	0.35	0.2
generate alignments	0.2	1	0.6
revise alignments	0.5	1.5	0.9
total	0.9	2.7	185

TABLE II. PRODUCTIVITY ESTIMATES OBTAINED WHEN CONVENTIONAL CODING IS APPLIED IN THE IN THE CALENDAR INTEGRATION TASKS..

Task	Minimum p. h.	Maximum p. h.	Average p. h.
Learning phase	8	22	14
coding and test	1	20	11
total	11	41	25

TABLE III. PERSON HOURS FOR CREATING THE THREE DATA INTEGRATION STEPS FOR THE POS PRINTING TASK.

Task	revise alignments (p. h.)	Create global ontology (p. h.)	Create local ontology (p. h.)
POS Printing	21	8	0.2

V. SUMMARY

Here, a framework was described for system integration, mainly intended for data integration among systems. The framework contains some components that are freely downloadable, and the majority of them are still being developed (these available solutions, like D2RQ, Quest and Alignapi were mentioned earlier, and in the references). These components require further modifications that are essential for better data integration. We presented novel methods and showed how to apply them. Some parts of our novel methods had to be implemented because some published algorithms are still not available for download (e.g. SPARQL-RW). In the future, we would like to develop our framework further by adding a GUI that supports the semi-automatic control of all the processes.

VI. ACKNOWLEDGEMENTS

This work was partially supported by the European Union and the European Social Fund through project FuturICT.hu (grant no.: TAMOP-4.2.2.C-11/1/KONV-2012-0013). Also, the work was supported by the

European Union, co-financed by the European Regional Development Fund (grant no.: GOP-1.1.1-11-2011-0052).

VII. REFERENCES

- [1] B. C. Grau, I. Horrocks, B. Motik, B. Parsia, P. Patel-Schneider, and U. Sattler, "OWL 2: The next step for OWL," *J. of Web Semantics*, vol. 6, no. 4, pp. 309–322, Nov. 2008.
- [2] I. Horrocks, "OWL: a description logic based ontology language.," in *Proc. of Int. Conf. on Principles and Practice of Constraint Programming (CP 2005)*, 2005, vol. 3709, pp. 5–8.
- [3] J. Pérez, M. Arenas, and C. Gutierrez, "Semantics and complexity of SPARQL," *ACM Trans. Database Syst.*, vol. 34, no. 3, pp. 16:1–16:45, 2009.
- [4] Y. Arens, C.-N. Hsu, and C. A. Knoblock, "Query processing in the SIMS information mediator," in *Readings in agents*, M. N. Huhns and M. P. Singh, Eds. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1998, pp. 82–90.
- [5] H. Wache, T. Vögele, U. Visser, H. Stuckenschmidt, G. Schuster, H. Neumann, and S. Hübner, "Ontology-based integration of information - A survey of existing approaches," 2001, pp. 108–117.
- [6] C. Bizer, "D2R MAP - A Database to RDF mapping language," in *Proceedings of the 12th International World Wide Web Conference (WWW 2003)*, 2003.
- [7] A. Das Sarma, X. Dong, and A. Halevy, "Bootstrapping pay-as-you-go data integration systems," in *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*, New York, NY, USA, 2008, pp. 861–874.
- [8] A. Buccella, A. Cechich, and N. R. Brisaboa, "Ontology-based data integration methods: A framework for comparison," *Revista Colombiana de Computación*, vol. 6, no. 1, 2005.
- [9] S. Suwanmanee, D. Benslimane, P.-A. Champin, and P. Thiran, "Wrapping and integrating heterogeneous databases with OWL," in *7th International Conference on Enterprise Information Systems, (ICIES 2005)*, 2005, pp. 11–18.
- [10] C. Bizer, "D2RQ - treating non-RDF databases as virtual RDF graphs," in *Proceedings of the 3rd International Semantic Web Conference (ISWC2004)*, 2004.
- [11] I. F. Cruz and H. Xiao, "The role of ontologies in data integration," *Journal of Engineering Intelligent Systems*, vol. 13, no. 4, 2005.
- [12] F. Priyatna, "RDF-based access to multiple relational data sources," *Universidad Politécnica de Madrid*, 2009.
- [13] B. Walshe, "Identifying complex semantic matches," in *Proceedings of the 9th international conference on The Semantic Web: research and applications*, Berlin, Heidelberg, 2012, pp. 849–853.
- [14] Konstantinos E. Makris, "SPARQL rewriting for query mediation over mapped ontologies," *Technical University of Crete*, 2010.
- [15] J. David, J. Euzenat, F. Scharffe, and C. Trojahn dos Santos, "The Alignment API 4.0," *Semant. web*, vol. 2, no. 1, pp. 3–10, Jan. 2011.
- [16] K. Makris, N. Bikakis, N. Gioldasis, and S. Christodoulakis, "SPARQL-RW: transparent query access over mapped RDF data sources.," in *EDBT*, 2012, pp. 610–613.
- [17] M. R. Muro, J. Hardi, and D. Calvanese, "Quest: efficient SPARQL-to-SQL for RDF and OWL," in *11th International Semantic Web Conference (ISWC2012)*.
- [18] C. Bizer, "D2RQ - treating non-RDF databases as virtual RDF graphs," in *Proceedings of the 3rd International Semantic Web Conference (ISWC2004)*, 2004.
- [19] M. Rodriguez-Muro and D. Calvanese, "Quest, an OWL 2 QL reasoner for ontology-based data access," in *OWLED*, 2012.
- [20] K. Makris, N. Gioldasis, N. Bikakis, and S. Christodoulakis, "Ontology mapping and SPARQL rewriting for querying federated RDF data sources," in *Proceedings of the 2010 international conference on the move to meaningful internet systems: Part II*, Berlin, Heidelberg, 2010, pp. 1108–1117.